

Vietnamese Restaurant Chain Mobile Ordering App

Work Breakdown Structure

Scope Mode | 5 Locations | iOS & Android

Date: 2026-03-10 | Version: 1.0

Prepared by: ZenLabs

Executive Summary

This Work Breakdown Structure defines the scope for building a cross-platform mobile ordering application for a Vietnamese restaurant chain operating 5 locations. The system consists of two primary interfaces: a customer-facing mobile app (iOS and Android) for menu browsing, ordering, online payment, and loyalty points; and a staff-facing tablet dashboard for real-time order management. The recommended technical approach is React Native for cross-platform mobile development, a NestJS modular monolith backend with PostgreSQL and Redis, Stripe for payment processing, and Socket.IO for real-time order updates. Key risks include payment gateway compliance (PCI-DSS), real-time reliability in restaurant WiFi environments, and menu customization complexity inherent to Vietnamese cuisine. The project is scoped at module level across all SDLC phases.

Technical Feasibility

Each major requirement has been assessed for technical feasibility. Items marked as 'Needs POC' require a technical spike before committing to the approach.

Requirement	Feasibility	Notes
Cross-platform mobile app (iOS + Android)	Viable	React Native is mature; Stripe and push notification SDKs well-supported on both platforms.
Online payment processing	Viable	Stripe handles PCI compliance via tokenization. Apple Pay and Google Pay supported through Stripe SDK.
Real-time order updates to staff tablets	Viable	WebSocket/Socket.IO with NestJS. Redis pub/sub for multi-instance scaling. Needs reconnection logic for spotty WiFi.
Loyalty points system	Viable	Custom-built. Standard points-per-dollar model with tier thresholds. No third-party dependency needed.
Multi-location menu management	Needs POC	Vietnamese cuisine has complex modifiers (pho options, spice levels). Need to validate data model handles per-location availability and customization.
POS system integration	Needs POC	Depends on existing POS vendor. POS APIs vary widely in quality. May need middleware layer. Recommend deferring to Phase 2 if no POS exists.
Offline resilience for staff tablets	Viable	Service worker / local queue for order display. Critical orders cached locally. Sync on reconnect.

Approach Analysis

Mobile Framework

Approach	Quality	Time	Budget	Recommendation
Native (Swift + Kotlin)	Best UX, full SDK access	2x development effort, requires 2 teams	High	Overkill for this scope
React Native	Good UX, mature ecosystem, strong Stripe/push SDKs	Single codebase, large talent pool	Moderate	RECOMMENDED
Flutter	Excellent UX, fast dev	Single codebase, growing ecosystem	Moderate	Strong alternative, smaller talent pool

Backend Architecture

Approach	Quality	Time	Budget	Recommendation
Modular Monolith (NestJS)	Clean separation, easy deployment, WebSocket support built-in	Fast to build, single deployment target	Low	RECOMMENDED
Microservices	Maximum scalability, independent deploys	Significant overhead, complex orchestration	High	Overengineered for 5 locations
BaaS (Firebase/Supabase)	Real-time built-in, auth included	Fastest to MVP	Variable	Vendor lock-in risk, scaling cost unclear

Database

Approach	Fit	Recommendation
PostgreSQL + Redis	ACID transactions for payments/orders, relational integrity, Redis for caching and pub/sub	RECOMMENDED
MongoDB	Flexible schema but weak for transactional order/payment data	Not recommended
Firebase Firestore	Real-time sync built-in but complex queries and vendor lock-in	Not recommended

Other Key Decisions

Decision Area	Chosen Approach	Rationale
Payment Processing	Stripe	PCI compliance handled, Apple Pay/Google Pay support, mature React Native SDK, refund handling built-in
Real-Time Updates	Socket.IO via NestJS	Native NestJS support, Redis adapter for horizontal scaling, robust reconnection handling
Loyalty Program	Custom-built	Simple enough for custom implementation, deeply tied to order flow, avoids third-party dependency costs
Push Notifications	Firebase Cloud Messaging	Free tier sufficient, cross-platform support, React Native libraries mature

AI Acceleration Opportunities

- **Claude Code:** Generate NestJS module scaffolding, React Native screen boilerplate, API contracts, database migrations, unit test stubs
- **Diffy Workflows:** Automate order notification flows, customer re-engagement campaigns, loyalty milestone alerts
- **AI-Assisted QA:** Generate test cases from OpenAPI specs, automated accessibility audits for mobile screens

Work Breakdown Structure

Phase 1: Discovery & Requirements

Module	Scope	Key Deliverables	Dependencies	Risk
Requirements Documentation	Formalize all functional and non-functional requirements from stakeholder interviews. Define user personas for customers and staff.	PRD document, user personas, acceptance criteria per feature	None (starting point)	Low
User Story Mapping	Map user journeys for: customer ordering flow, staff order management flow, loyalty enrollment and redemption.	User story map, prioritized backlog, MVP scope definition	Requirements Documentation	Low

Module	Scope	Key Deliverables	Dependencies	Risk
Technical Spike: Menu Data Model	Prototype the menu data model to handle Vietnamese cuisine complexity (modifiers, variations, per-location availability).	POC data model, sample API responses, stakeholder sign-off	Requirements Documentation	Medium
Technical Spike: POS Integration	Evaluate existing POS system API capabilities. Determine integration feasibility or standalone approach.	POS API assessment, integration recommendation, fallback plan	Requirements Documentation	Medium

Phase 2: System Design & Architecture

Module	Scope	Key Deliverables	Dependencies	Risk
High-Level Architecture	Define system architecture: React Native apps, NestJS backend, PostgreSQL, Redis, Stripe integration, Socket.IO real-time layer.	Architecture diagram, technology stack document, deployment topology	Discovery phase complete	Low
Data Model Design	Design database schema covering: users, locations, menus, menu items, modifiers, orders, order items, payments, loyalty accounts, loyalty transactions.	ERD diagram, migration scripts, seed data strategy	High-Level Architecture, Menu POC	Medium
API Contract Design	Define RESTful API contracts for all modules. OpenAPI 3.0 spec. WebSocket event contracts for real-time order updates.	OpenAPI spec, WebSocket event catalog, Postman collection	Data Model Design	Low
Security Architecture	Design auth flow (JWT + refresh tokens), role-based access (customer vs staff), Stripe webhook verification, rate limiting, input validation strategy.	Security design doc, auth flow diagrams, threat model	High-Level Architecture	Medium
Infrastructure Blueprint	Define hosting (cloud provider), CI/CD pipeline, environment strategy (dev/staging/prod), monitoring and alerting setup.	Infrastructure diagram, IaC templates, cost estimate	High-Level Architecture	Low

Phase 3: Development

Backend Modules

Module	Scope	Key Deliverables	Dependencies	Risk
Authentication Module	User registration (email/phone), login, JWT issuance and refresh, password reset, social login (optional). Separate roles for customer and staff.	Auth endpoints, JWT middleware, role guard, session management	Data Model, Security Architecture	Low
Location Module	CRUD for restaurant locations. Store hours, address, geolocation, contact info. Location-based filtering for menus and orders.	Location API, geolocation-based store finder, admin management UI endpoints	Authentication Module	Low
Menu Management Module	Menu categories, items, modifiers (toppings, spice level, protein choice), pricing, images, per-location availability, item status (available/sold out).	Menu CRUD API, modifier system, availability toggles, image upload	Location Module, Menu POC	Medium

Module	Scope	Key Deliverables	Dependencies	Risk
Order Module	Cart management, order placement, order status lifecycle (placed, confirmed, preparing, ready, picked up), order history. Supports modifiers and special instructions.	Order API, status machine, cart persistence, order history endpoints	Menu Module, Authentication Module	Medium
Payment Module	Stripe integration for card payments, Apple Pay, Google Pay. Payment intent creation, webhook handling, refund processing, receipt generation, tip support.	Payment API, Stripe webhook handler, refund flow, receipt emails	Order Module	High
Loyalty Module	Points accrual per order (configurable rate), points balance, tier calculation, reward catalog, reward redemption, points history, promotional bonus points.	Loyalty API, points engine, tier logic, reward redemption flow	Order Module, Payment Module	Medium
Real-Time Notification Module	WebSocket gateway for staff tablets (new order alerts, status updates). Push notifications to customers (order status changes, loyalty milestones). FCM integration.	Socket.IO gateway, FCM integration, notification templates, sound alerts	Order Module	Medium
Admin Module	Dashboard for chain owner: sales reporting, location management, menu management, loyalty program configuration, staff account management.	Admin API endpoints, reporting queries, configuration endpoints	All other backend modules	Low

Frontend Modules (React Native)

Module	Scope	Key Deliverables	Dependencies	Risk
Customer App: Onboarding	Registration, login, phone verification, location permission request, store selection.	Auth screens, onboarding flow, store selector with map	Auth API	Low
Customer App: Menu Browsing	Category navigation, item detail with images, modifier selection, search and filter, dietary tags.	Menu screens, item detail with modifier picker, search UI	Menu API	Low
Customer App: Cart & Checkout	Cart management, order review, Stripe payment sheet, tip selection, special instructions, order confirmation.	Cart screen, checkout flow, Stripe payment integration, order confirmation	Order API, Payment API	Medium
Customer App: Order Tracking	Real-time order status display, push notification handling, order history with reorder capability.	Order status screen, push notification handlers, order history	Real-Time Notification Module	Medium
Customer App: Loyalty	Points balance display, tier progress, reward catalog, redemption flow, points history.	Loyalty dashboard, reward catalog screen, redemption flow	Loyalty API	Low
Staff Tablet App: Order Dashboard	Real-time order feed with audio alerts, order detail view, status update controls (confirm, preparing, ready), sold-out item toggle.	Order dashboard with real-time updates, status controls, alert system	Real-Time Notification Module, Order API	High
Staff Tablet App: Management	Daily sales summary, menu item availability toggles, order history search, basic reporting.	Management screens, availability controls, daily summary view	Admin API	Low

Phase 4: Quality Assurance

Module	Scope	Key Deliverables	Dependencies	Risk
Unit & Integration Testing	Unit tests for all backend modules (Jest). Integration tests for API endpoints. Target: 80%+ coverage on business logic.	Test suites, coverage reports, CI pipeline integration	All development modules	Low
End-to-End Testing	E2E tests for critical flows: ordering, payment, loyalty redemption, real-time order updates. Detox for React Native.	E2E test suite, test environment, automated regression	Unit testing complete	Medium
Payment Testing	Stripe test mode validation: successful payments, declined cards, refunds, webhook failures, Apple Pay sandbox, Google Pay sandbox.	Payment test matrix, edge case validation, PCI compliance checklist	Payment Module	High
Performance & Load Testing	Simulate peak ordering (lunch/dinner rush across 5 locations). WebSocket connection limits. Database query performance under load.	Load test scripts, performance benchmarks, optimization recommendations	All modules deployed to staging	Medium
UAT & Accessibility	User acceptance testing with restaurant staff and sample customers. Accessibility audit (WCAG 2.1 AA). Usability testing on actual tablets.	UAT sign-off, accessibility report, usability findings	E2E testing complete	Low

Phase 5: Deployment & DevOps

Module	Scope	Key Deliverables	Dependencies	Risk
CI/CD Pipeline	Automated build, test, and deployment pipeline. Separate flows for backend (NestJS) and mobile (React Native). App store submission automation.	CI/CD configuration, build scripts, deployment runbooks	Infrastructure Blueprint	Low
Environment Setup	Provision dev, staging, and production environments. Database provisioning, Redis clusters, SSL certificates, domain configuration.	Environment configs, IaC scripts, environment parity validation	Infrastructure Blueprint	Low
App Store Submission	Apple App Store and Google Play Store submission. Store listings, screenshots, privacy policy, review compliance.	Store listings, app review approval, launch readiness checklist	UAT complete, CI/CD pipeline	Medium
Monitoring & Alerting	Application monitoring (errors, performance), infrastructure monitoring, business metrics dashboards, on-call alerting for critical failures.	Monitoring dashboards, alert rules, incident response playbook	Environment Setup	Low

Phase 6: Post-Launch & Maintenance

Module	Scope	Key Deliverables	Dependencies	Risk
Launch Support	Phased rollout strategy (1 location first, then all 5). On-site support during initial launch. Staff training sessions. Customer support FAQ.	Rollout plan, training materials, support documentation	All prior phases complete	Medium
Post-Launch Monitoring	Intensive monitoring period (2 weeks). Bug triage and hotfix process. Performance tuning based on real usage data.	Monitoring reports, bug fix releases, performance tuning	Launch Support	Low
Knowledge Transfer	Technical documentation handoff. Codebase walkthrough. Operational runbooks. Support escalation procedures.	Technical docs, video walkthroughs, operational runbooks	Post-Launch Monitoring	Low
Phase 2 Roadmap	Define next-phase features: delivery integration, table reservation, advanced analytics, POS deep integration, multi-language support, promotional campaigns.	Phase 2 feature backlog, prioritization matrix, technical feasibility notes	Post-Launch Monitoring	Low

Dependency Map

The following table shows module dependencies and the critical path through the project. Modules on the critical path (marked with *) determine the minimum project timeline.

Module	Depends On	Blocks	Critical Path
Requirements Documentation	(none)	All design modules	Yes *
High-Level Architecture	Requirements	All design modules	Yes *
Data Model Design	Architecture, Menu POC	All backend modules	Yes *
API Contract Design	Data Model	Frontend modules	Yes *
Authentication Module	Data Model, Security Arch	All other backend modules	Yes *
Menu Management Module	Location Module, Menu POC	Order Module, Frontend Menu	Yes *
Order Module	Menu Module, Auth Module	Payment, Loyalty, Notifications	Yes *
Payment Module	Order Module	Loyalty Module	Yes *
Loyalty Module	Order, Payment Modules	Customer App Loyalty	No
Real-Time Notification Module	Order Module	Staff Dashboard, Order Tracking	Yes *
Staff Tablet Dashboard	Real-Time Module, Order API	UAT	Yes *
Customer App: Cart & Checkout	Order API, Payment API	E2E Testing	Yes *
E2E Testing	All dev modules	UAT	No
App Store Submission	UAT, CI/CD	Launch	Yes *

Risk Register

Risk	Likelihood	Impact	Mitigation Strategy
Payment processing failures during peak hours	Medium	High	Implement Stripe webhook retry logic, idempotent payment operations, fallback error handling with clear user messaging. Monitor payment success rate.
Real-time WebSocket disconnections in restaurant WiFi	High	High	Implement aggressive reconnection with exponential backoff. Audio alert on disconnect. Local order queue with sync-on-reconnect. Consider dedicated WiFi for tablets.
Menu data model insufficient for Vietnamese cuisine complexity	Medium	Medium	Technical spike in Phase 1 to validate data model. Prototype with real menu from one location. Iterate before committing to schema.
App Store rejection (Apple/Google)	Low	High	Follow platform guidelines from day one. Review checklist before submission. Allow 2-week buffer for review cycles.
POS integration complexity exceeds estimate	Medium	Medium	Defer POS integration to Phase 2 if assessment reveals high complexity. Design order module to work standalone with optional POS sync.
Scope creep from stakeholder feature requests	High	Medium	Strict MVP scope defined in Discovery phase. Change request process with impact assessment. Phase 2 roadmap absorbs non-critical requests.
Multi-location data inconsistency	Low	Medium	Centralized database with location-scoped queries. Transaction isolation for orders and payments. Automated consistency checks.

Assumptions & Constraints

Assumptions

- The restaurant chain operates in a single country/region with one currency and tax jurisdiction.
- Each location has reliable WiFi infrastructure for staff tablet connectivity.
- An existing brand identity (logo, colors, menu photography) is available for the customer-facing app.
- The restaurant does not currently have a POS system that requires integration (or POS integration can be deferred to Phase 2).
- Menu content (item names, descriptions, images, prices) will be provided by the client before development begins.
- The loyalty program follows a simple points-per-dollar model without complex promotional rules at launch.
- Pickup only at launch (no delivery integration in Phase 1).
- The client has Apple Developer and Google Play Developer accounts (or will create them).

Constraints

- Must support both iOS (14+) and Android (10+) platforms.
- Payment processing must be PCI-DSS compliant (achieved through Stripe tokenization).
- Real-time order updates must have less than 3-second latency under normal conditions.
- The system must handle concurrent ordering from all 5 locations without performance degradation.
- All customer data must be stored securely with encryption at rest and in transit.
- The app must meet WCAG 2.1 AA accessibility standards.